

ETL TESTING PROJECT – E-COMMERCE

Project objective:

To validate the extraction, transformation, and loading process of customers, orders and product data from source tables into target table and ensuring the data is accurate, complete and error free using ETL testing validation.

Technology Stack:

Source Database : MySQL

Target Database : Snowflake

Language : SQL

Automation : Python

Reporting : Excel

Tool : VS Code

Version Control : GitHub

Project Flow Chart:

Requirement Analysis



Mapping Document



CSV Files

(Customer, Product, Orders)



MySQL Source Tables



Transformation Logic

(JOIN & Data Cleansing)



Snowflake Target Table



ETL Validations

1. Record Count Validation

2. Null Validation
3. Duplicate Validation
4. Aggregate Validation
5. Data Comparison Validation



Python Automation



Excel Report



GitHub Upload

BRD (Requirement Analysis) – E-Commerce ETL Testing Project

Objective

Create a consolidated E-Commerce Dashboard containing:

Customer Information

Product Information

Order Information

Sales Information

Business Requirements

BR ID

Requirement

BR001

Load Customer Data

BR002

Load Product Data

BR003

Load Orders Data

BR004

Join Customer, Product and Orders datasets

BR005

Create E-Commerce Dashboard

BR006

No Duplicate Order ID

BR007

No NULL Order ID

BR008

Validate Total Orders Count

BR009

Validate Total Sales Amount

BR010

Validate Customer and Product Data Accuracy

Source Datasets:

customers.csv

products.csv

orders.csv

Expected Output:

Create a target table containing:

Customer id

Product id

Order ID

Order Date

Quantity

Price

Total Amount

DATABASE

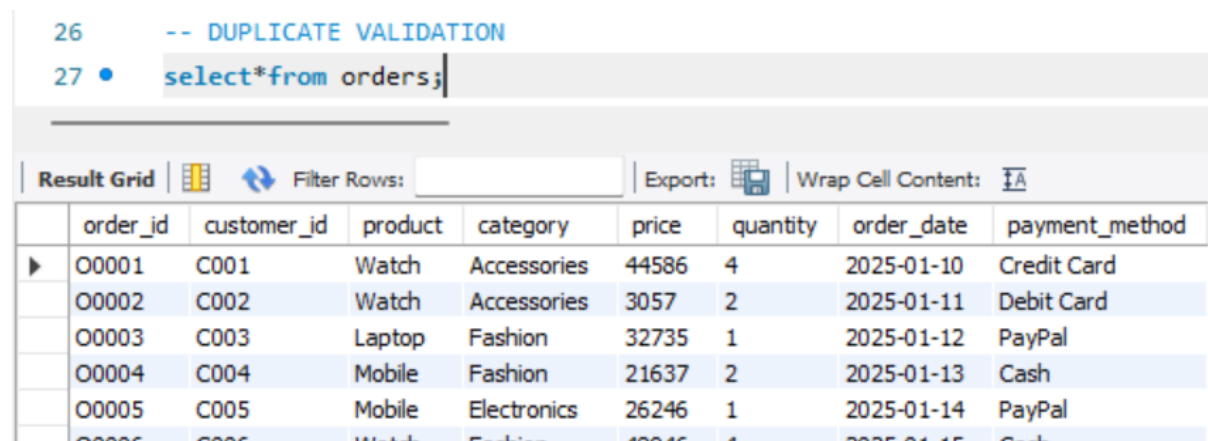
```
CREATE DATABASE ETL_PROJECT;
```

```
USE ETL_PROJECT;
```

SOURCE TABLE

1.ORDERS TABLE

```
CREATE TABLE orders(  
    order_id VARCHAR(10) PRIMARY KEY,  
    customer_id VARCHAR(10),  
    product VARCHAR(100),  
    category VARCHAR(50),  
    price DECIMAL(10,2),  
    quantity INT,  
    order_date DATE,  
    payment_method VARCHAR(50)  
)
```



The screenshot shows a SQL IDE interface. At the top, a query is entered in a text area:

```
26      -- DUPLICATE VALIDATION  
27 •    select*from orders;
```

Below the query editor, there is a toolbar with options: "Result Grid" (selected), "Filter Rows:", "Export:", and "Wrap Cell Content:". Below the toolbar, a table of results is displayed:

	order_id	customer_id	product	category	price	quantity	order_date	payment_method
▶	O0001	C001	Watch	Accessories	44586	4	2025-01-10	Credit Card
	O0002	C002	Watch	Accessories	3057	2	2025-01-11	Debit Card
	O0003	C003	Laptop	Fashion	32735	1	2025-01-12	PayPal
	O0004	C004	Mobile	Fashion	21637	2	2025-01-13	Cash
	O0005	C005	Mobile	Electronics	26246	1	2025-01-14	PayPal
	O0006	C006	Watch	Fashion	48046	4	2025-01-15	Cash

2.TARGET TABLE

```
CREATE TABLE orders_target AS
```

```
SELECT
```

```
order_id,
```

```
customer_id,
```

```
UPPER(product) AS product_name,  
UPPER(category) AS category_name,  
price,  
quantity,  
(price * quantity) AS total_amount,  
order_date,  
payment_method  
FROM orders  
WHERE order_id IS NOT NULL;
```

3. TRANSFORMATION LOGIC:

Business Requirement:

Combine Orders data and calculate total sales amount for reporting.

Transformation Query

```
CREATE TABLE orders_target AS SELECT  
    order_id,  
    customer_id,  
    product,  
    category,  
    price,  
    quantity,  
    (price * quantity) AS total_amount,  
    order_date,  
    payment_method  
FROM orders;
```

Transformation Applied:

- Extract data from orders source table.

- Calculate Total Amount = Price × Quantity.
- Load transformed data into orders_target table.
- Create a consolidated reporting table for ETL validation.

VALIDATIONS

SOURCE TABLE

-- RECORD COUNT VALIDATION

```
select count(*) as total_records from orders;
```

-- NULL VALIDATION

```
select *from orders where order_id is null;
```

-- DUPLICATE VALIDATION

```
select*from orders;
```

```
select order_id,count(*) from orders
```

```
group by order_id having count(*)>1;
```

-- AGGREGATE VALIDATION

```
select count(*) from orders;
```

```
select count(distinct order_id)from orders;
```

```
select sum(price) from orders;
```

```
select avg(price) as avg_price from orders;
```

```
select max(price) as max_price from orders;
```

```
select min(price) as min_price from orders;
```

-- UNIQUE VALIDATION

```
select count(order_id),('total_ids'),
```

```
count(distinct order_id)as unique_ids from orders;
```

-- DATATYPE VALIDATION

-- PRODUCT NAME UPPERCASE

```
select order_id,upper(product) as product_name from orders;
```

-- CATOGORY UPPERCASE

select upper(category) as category_name from orders;

-- TOTAL AMOUNT CALCULATION

select order_id,price,quantity,
(price*quantity)as total_amount from orders;

-- TOTAL AMOUNT CALCULATION

select order_id,price,quantity,
(price*quantity)as total_amount from orders;

-- TARGET VALIDATION

select count(*) from orders;

select count(*) from orders_target;

--NULL VALIDATION

SELECT*FROM ORDERS_TARGET

WHERE ORDER_ID IS NULL;

-- DUPLICATE VALIDATION

SELECT ORDER_ID,COUNT(*)

FROM ORDERS_TARGET

GROUP BY ORDER_ID

HAVING COUNT(*)>1;

--AGGREGATE VALIDATION

SELECT SUM(TOTAL_AMOUNT) as total_sales from orders_target;

RECORD COUNT VALIDATION

-- ORDERS SOURCE COUNT

select count(*) as source_count from orders;

-- ORDERS TARGET COUNT

```
select count(*) as target_count from orders;
```

-- ORDERS NULL VALIDATION

```
select count(*) from orders
```

```
where order_id is null;
```

```
select count(*) from orders
```

```
where customer_id is null;
```

DUPLICATE VALIDATION

```
select order_id,count(*) from orders
```

```
group by order_id having count(*)>1;
```

##AGGREGATE VALIDATION

##AGGREGATE VALIDATION

-- price

```
select sum(price)as total_price from orders;
```

-- quantity

```
select SUM(QUANTITY)AS TOTAL_QUANTITY FROM ORDERS;
```

DATA COMPARISION VALIDATION

-- RECORD BY RECORD COMPARISION

```
select *from orders s left join orders_target t
```

```
on s.order_id = t.order_id where t.order_id is null;
```

SCREENSHOTS


```
15 • SELECT COUNT(*) from orders;
16
17
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	COUNT(*)			
▶	20			

```
18 ## MYSQL SOURCE VALIDATION
19
20 -- RECORD COUNT VALIDATION
21 • select count(*) as total_records from orders;
22
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_records			
▶	20			

```
23 -- NULL VALIDATION
24 • select *from orders where order_id is null;
25
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

order_id	customer_id	product	category	price	quantity	order_date	payment_method
----------	-------------	---------	----------	-------	----------	------------	----------------

```

18      ## MYSQL SOURCE VALIDATION
19
20      -- RECORD COUNT VALIDATION
21 •    select count(*) as total_records from orders;
22

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	total_records
▶	20

```

23      -- NULL VALIDATION
24 •    select *from orders where order_id is null;
25

```

Result Grid   Filter Rows: Export:  Wrap Cell Content:  

	order_id	customer_id	product	category	price	quantity	order_date	payment_method
--	----------	-------------	---------	----------	-------	----------	------------	----------------

orders 5  

```

26      -- DUPLICATE VALIDATION
27 •    select*from orders;
28 •    select order_id,count(*) from orders
29      group by order_id having count(*)>1;
30

```

Result Grid   Filter Rows: Export:  Wrap Cell Content:  

	order_id	count(*)
--	----------	----------

```
36 • select sum(price) from orders;
37
38 • select avg(price) as avg_price from orders;
39
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
sum(price)			
▶ 511981			

```
38 • select avg(price) as avg_price from orders;
39
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
avg_price			
▶ 25599.0500			

Result 10 

```
40 • select max(price) as max_price from orders;
41
42 • select min(price) as min_price from orders;
43
```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content:  

	max_price
▶	49415

```
42 • select min(price) as min_price from orders;
43
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	min_price			
▶	3057			

```
47 -- UNIQUE VALIDATION
48 • select count(order_id),('total_ids'),
49 count(distinct order_id)as unique_ids from orders;
50
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	count(order_id)	total_ids	unique_ids	
▶	20	total_ids	20	

```
51 ## TRANSFORMATION LOGIC
52
53 -- PRODUCT NAME UPPERCASE
54 • select order_id,upper(product) as product_name from orders;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	order_id	product_name		
	O0001	WATCH		
	O0002	WATCH		
	O0003	LAPTOP		
	O0004	MOBILE		
	O0005	MOBILE		
	O0006	WATCH		

```

56      -- CATEGORY UPPERCASE
57 •    select upper(category) as category_name from orders;
58
59      -- TOTAL AMOUNT CALCULATION
60 •    select order_id,price,quantity,

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	category_name
▶	ACCESSORIES
	ACCESSORIES
	FASHION
	FASHION
	ELECTRONICS
	FASHION

```

59      -- TOTAL AMOUNT CALCULATION
60 •    select order_id,price,quantity,
61      (price*quantity)as total_amount from orders;
62
63      -- REMOVE NULL RECORDS

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	order_id	price	quantity	total_amount
▶	O0001	44586	4	178344
	O0002	3057	2	6114
	O0003	32735	1	32735
	O0004	21637	2	43274
	O0005	26246	1	26246
	O0006	48946	4	195784

```

82      -- TARGET VALIDATION
83 •    select count(*) from orders;
84 •    select count(*) from orders_target;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	count(*)
▶	20

```
86     ## NULL VALIDATION
87     • SELECT*FROM ORDERS_TARGET
88     WHERE ORDER_ID IS NULL;
89
```

Result Grid   Filter Rows: Export:  Wrap Cell Content:  									
order_id	customer_id	product_name	category_name	price	quantity	total_amount	order_date	paymer	

```
90      ## DUPLICATE VALIDATION
91      SELECT ORDER_ID,COUNT(*)
92      FROM ORDERS_TARGET
93      GROUP BY ORDER_ID
94      HAVING COUNT(*)>1;
```

```
90  ## DUPLICATE VALIDATION
91  ●  SELECT ORDER_ID,COUNT(*)
92     FROM ORDERS_TARGET
93     GROUP BY ORDER_ID
94     HAVING COUNT(*)>1;
```

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 



	ORDER_ID	COUNT(*)
--	----------	----------

```
96      ## AGGREGATE VALIDATION
97 •    SELECT SUM(TOTAL_AMOUNT) as total_sales from orders_target;
98
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_sales			
	1045461			

```
105      -- ORDERS TARGET COUNT
106
107 •    select count(*) as target_count from orders;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	target_count			
	20			

```
120      ##AGGREGATE VALIDATION
121      -- price
122 •    select sum(price)as total_price from orders;
123
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_price			
	511981			

DATA COMPARISION VALIDATION

```
129     ## DATA COMPARISION VALIDATION
130     -- RECORD BY RECORD COMPARISION
131     • select *from orders s left join orders_target t
132       on s.order_id = t.order_id where t.order_id is null;
133
```

Result Grid



Filter Rows:

Export: 

Wrap Cell Content: 



order_id	customer_id	product	category	price	quantity	order_date	payment_method	order_id	cus
----------	-------------	---------	----------	-------	----------	------------	----------------	----------	-----

```
134 -- price comparision
135 • SELECT s.order_id,s.price,t.price
136 FROM orders s
137 JOIN orders_target t
138 ON s.order_id=t.order_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

```
141 -- QUANTITY COMPRISION
142 • SELECT s.order_id,s.quantity,t.quantity
143 FROM orders s
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

order_id	quantity	quantity
----------	----------	----------

PYTHON AUTOMATION

```
orders_validation.py ●
> Users > LAP-214 > Documents > etl project > orders_validation.py
1 |
2 import mysql.connector
3
4 conn = mysql.connector.connect(
5     host = "localhost",
6     user = "root",
7     password = "root",
8     database = "etl_project"
9 )
10 cursor = conn.cursor()
11
12 print("==== ORDERS VALIDATION REPORT ====")
13
14 # 1. Record Count Validation
15 cursor.execute("SELECT COUNT(*) FROM orders")
16 source_count = cursor.fetchone()[0]
17
18 cursor.execute("SELECT COUNT(*) FROM orders_target")
19 target_count = cursor.fetchone()[0]
20
21 print(f"\nSource Count : {source_count}")
22 print(f"Target Count : {target_count}")
23
24 if source_count == target_count:
25     print("Record Count Validation : PASS")
26 else:
27     print("Record Count Validation : FAIL")
28
```

```
# 2. Null Validation
cursor.execute("""
SELECT COUNT(*)
FROM orders
WHERE customer_id IS NULL
""")

null_count = cursor.fetchone()[0]

print(f"\nNull Count : {null_count}")

if null_count == 0:
    print("Null Validation : PASS")
else:
    print("Null Validation : FAIL")

# 3. Duplicate Validation
cursor.execute("""
SELECT COUNT(*)
FROM (
    SELECT order_
    FROM orders
    GROUP BY order_id
    HAVING COUNT(*) > 1
) a
""")
```

```
# 3. Duplicate Validation
cursor.execute("""
SELECT COUNT(*)
FROM (
    SELECT order_
    FROM orders
    GROUP BY order_id
    HAVING COUNT(*) > 1
) a
""")

dup_count = cursor.fetchone()[0]

print(f"\nDuplicate Count : {dup_count}")

if dup_count == 0:
    print("Duplicate Validation : PASS")
else:
    print("Duplicate Validation : FAIL")
```

```
# 4. Aggregate Validation - Price
cursor.execute("SELECT SUM(price) FROM orders")
source_price = cursor.fetchone()[0]

cursor.execute("SELECT SUM(price) FROM orders_target")
target_price = cursor.fetchone()[0]

print(f"\nSource Price Sum : {source_price}")
print(f"Target Price Sum : {target_price}")

if source_price == target_price:
    print("Price Aggregate Validation : PASS")
else:
    print("Price Aggregate Validation : FAIL")
```

```
# 5. Aggregate Validation - Quantity
cursor.execute("SELECT SUM(quantity) FROM orders")
source_qty = cursor.fetchone()[0]

cursor.execute("SELECT SUM(quantity) FROM orders_target")
target_qty = cursor.fetchone()[0]

print(f"\nSource Quantity Sum : {source_qty}")
print(f"Target Quantity Sum : {target_qty}")

if source_qty == target_qty:
    print("Quantity Aggregate Validation : PASS")
else:
    print("Quantity Aggregate Validation : FAIL")
```

```
# 6. Data Comparison Validation
cursor.execute("""
SELECT COUNT(*)
FROM orders s
LEFT JOIN orders_target t
ON s.order_id = t.order_id
WHERE t.order_id IS NULL
""")

mismatch_count = cursor.fetchone()[0]

print(f"\nMismatch Records : {mismatch_count}")

if mismatch_count == 0:
    print("Data Comparison Validation : PASS")
else:
    print("Data Comparison Validation : FAIL")

conn.close()

print("\n==== VALIDATION COMPLETED =====")
```